ГУАП

КАФЕДРА № 43

ОТЧЕТ
ЗАЩИЩЕН С ОЦЕНКОЙ

ПРЕПОДАВАТЕЛЬ

д-р техн. наук, доцент		С.И. Колесникова
должность, уч. степень, звание	подпись, дата	инициалы, фамилия

ОТЧЕТ О ЛАБОРАТОРНОЙ РАБОТЕ №4

Модели динамических систем-1

по курсу: КОМПЬЮТЕРНОЕ МОДЕЛИРОВАНИЕ

РАБОТУ ВЫПОЛНИЛ

СТУДЕНТ ГР. №	4236		А.В. Панченко
		подпись, дата	инициалы, фамилия

Санкт-Петербург 2025

Цель работ: Цель настоящей работы: освоить приемы моделирования непрерывных процессов в MatLab Simulink.

Вариант 19:

Вариант №19

1) $y''y^2 - 4y' = 0, \quad y(1) = 1, \quad y'(1) = -4$

2) $$\begin{cases} \dfrac{dx}{dt} = 2x + 3y \\ \dfrac{dy}{dt} = x + 4y \end{cases}, x(0) = 1, \quad y(0) = 0.$$

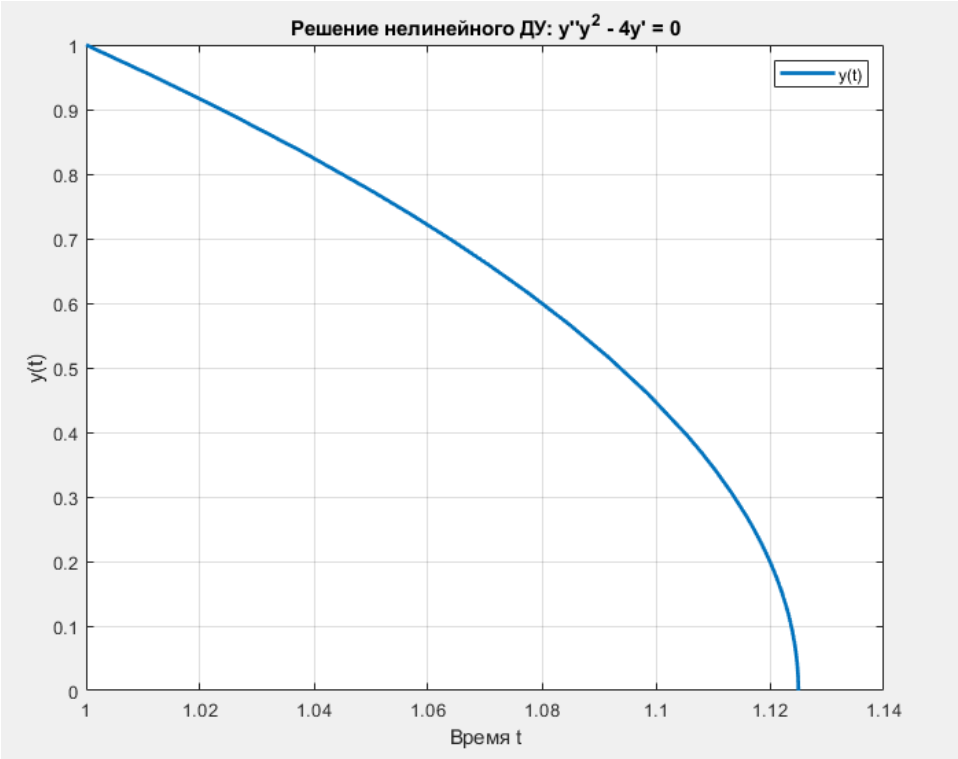
Ознакомление со справочными сведениями относительно приложения MatLab Simulink:

- Simulink – это интерактивное приложение в среде MATLAB, предназначенное для моделирования, анализа и проектирования динамических систем.
- Модели создаются в виде наглядных блок-схем (диаграмм) путем перетаскивания и соединения блоков из обширных библиотек.
- Библиотеки содержат блоки для источников сигналов (Source), приемников (Sink), непрерывных (Continuous) и дискретных (Discrete) систем, математических операций (Math Operations) и др.
- Основные этапы работы: создание модели, настройка параметров блоков и решателя (Solver), запуск моделирования, анализ результатов.

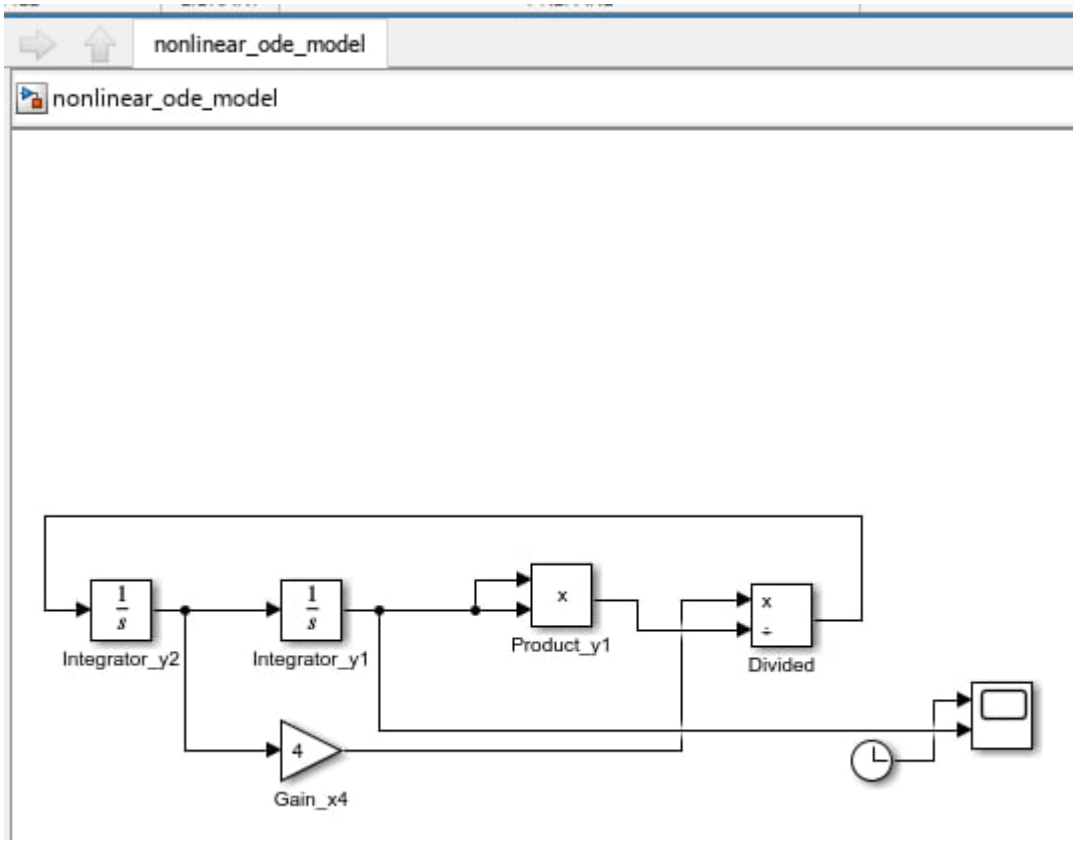
Построение графиков непрерывной нелинейной модели решения ДУ:

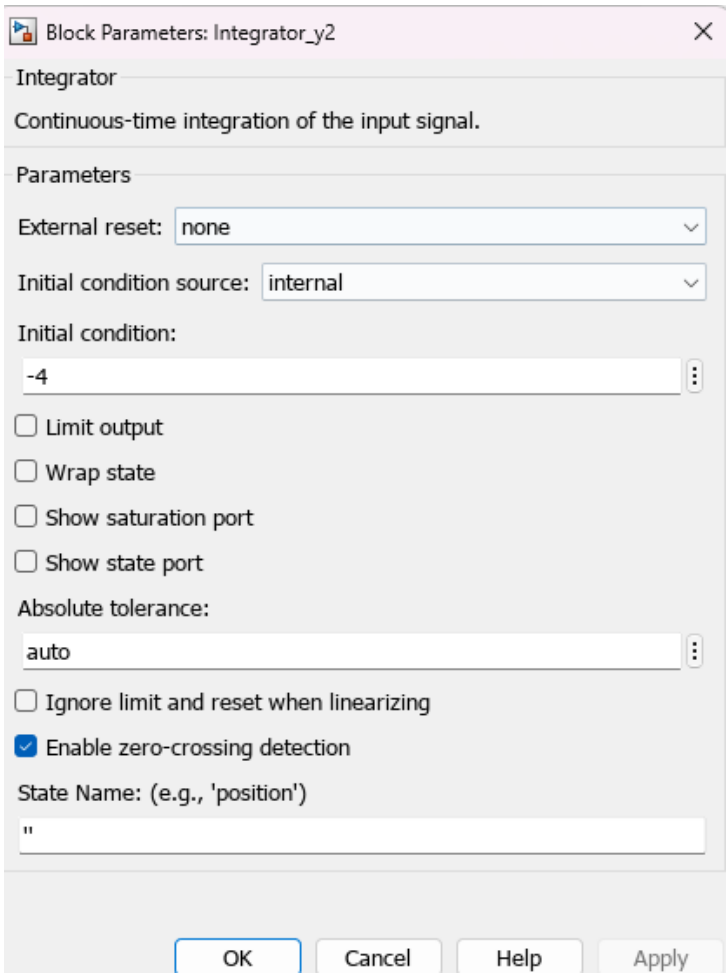
Код:

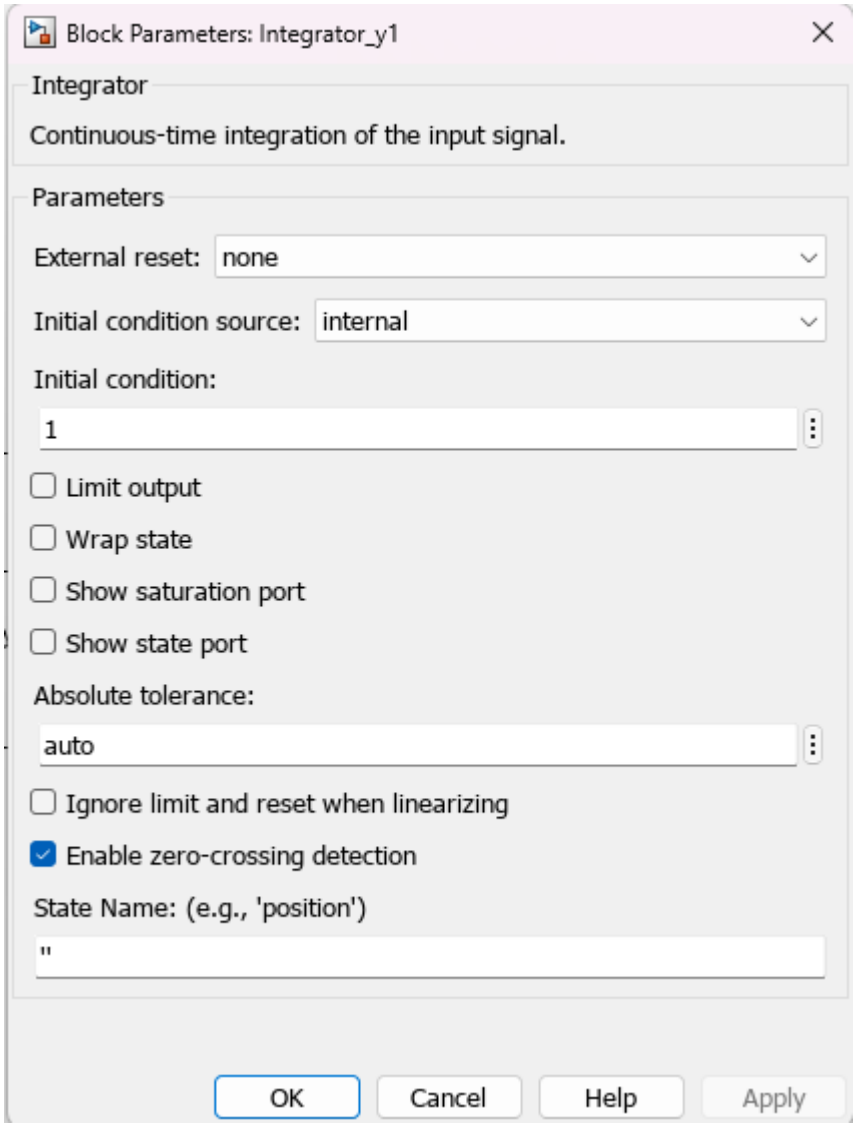
```
% Пункт 2. Решение нелинейного ДУ y''*y^2 - 4*y' = 0, y(1)=1, y'(1)=-4

% 1. Задаем временной интервал и начальные условия
tspan = [1 5]; % Время от 1 до 5 секунд
y0 = [1; -4];  % Вектор начальных условий [y(1); y'(1)]

% 2. Решаем систему ОДУ с помощью решателя ode45
[t, Y] = ode45(@odefun_nonlin, tspan, y0);

% 3. Извлекаем решение y(t) из результата
y = Y(:, 1); % Первый столбец - это y(t)

% 4. Строим график
figure;
plot(t, y, 'LineWidth', 2);
grid on;
title('Решение нелинейного ДУ: y''''y^2 - 4y'' = 0');
xlabel('Время t');
ylabel('y(t)');
legend('y(t)');

% 5. Функция, задающая систему ОДУ для решателя
function dydt = odefun_nonlin(t, y)
    % y(1) = y
    % y(2) = y'
    dydt = zeros(2,1);
    dydt(1) = y(2); % y' = y2
    dydt(2) = (4 * y(2)) / (y(1)^2); % y'' = (4*y') / (y^2)
end
```

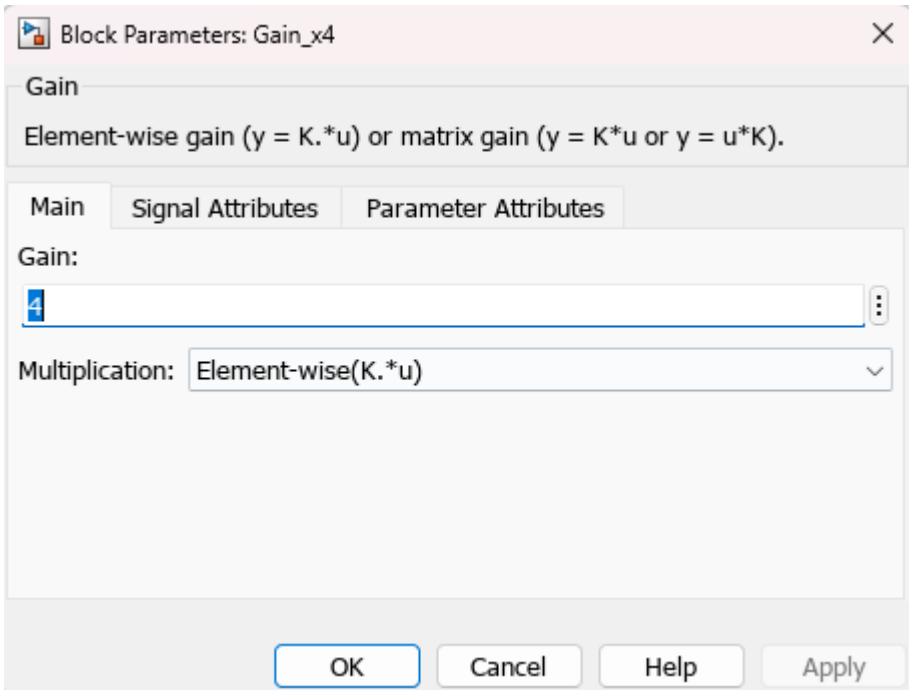


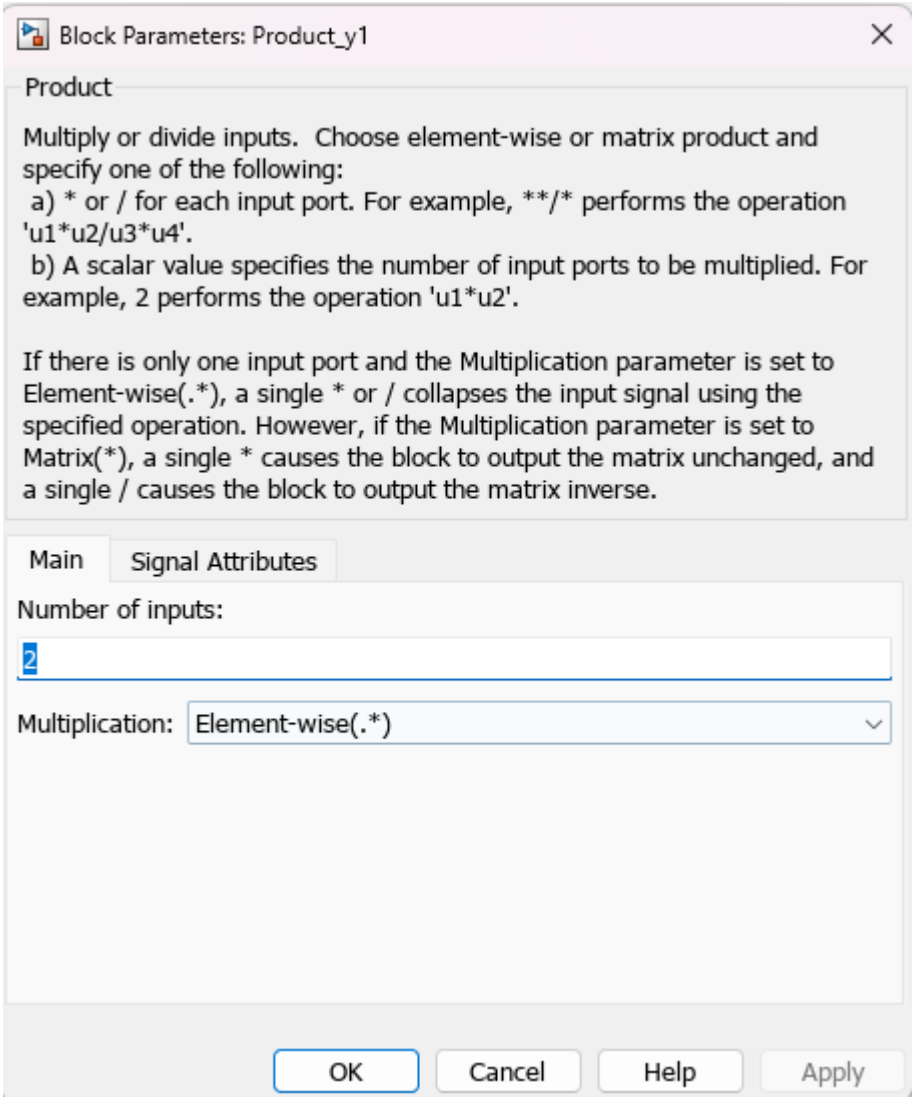
Разработка модели Simulink для решения нелинейного ДУ:

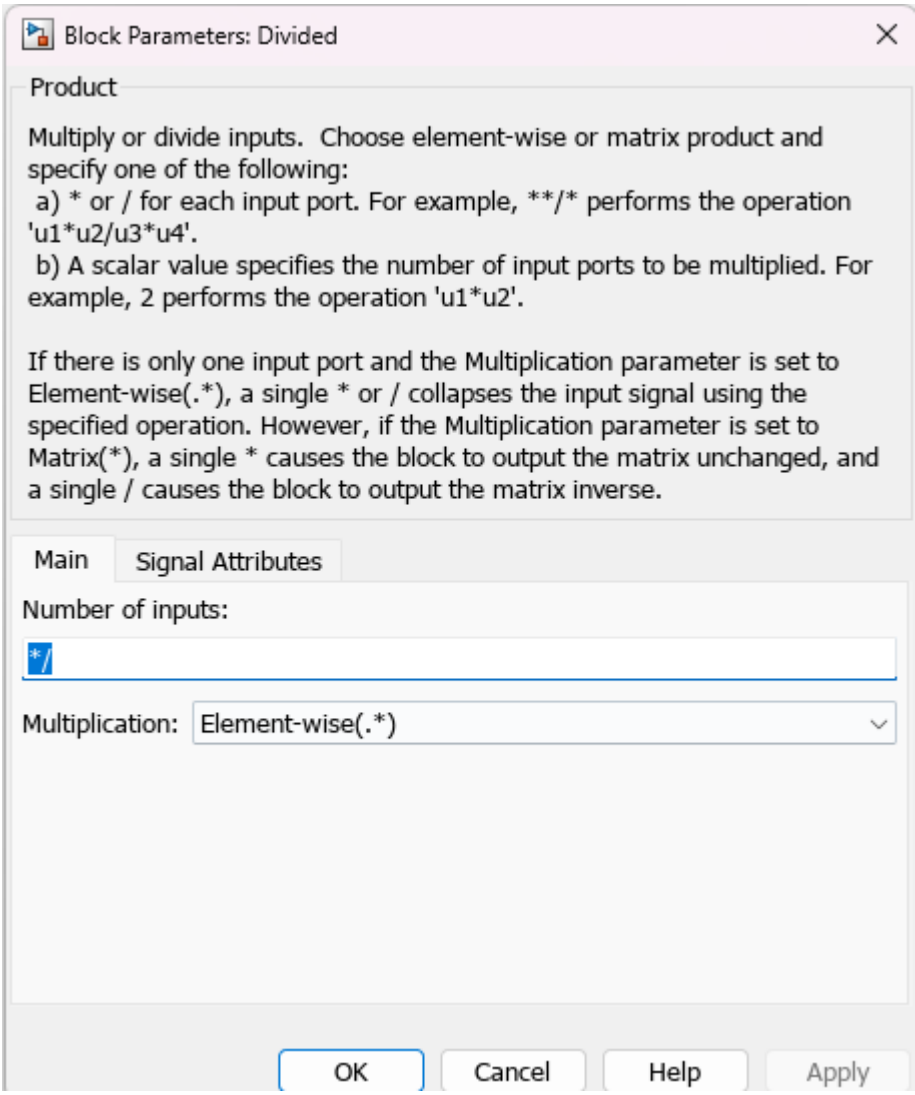


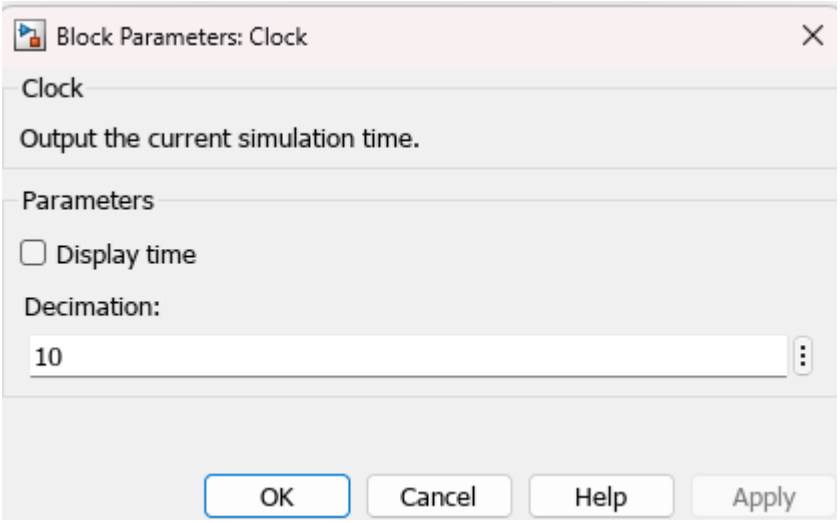


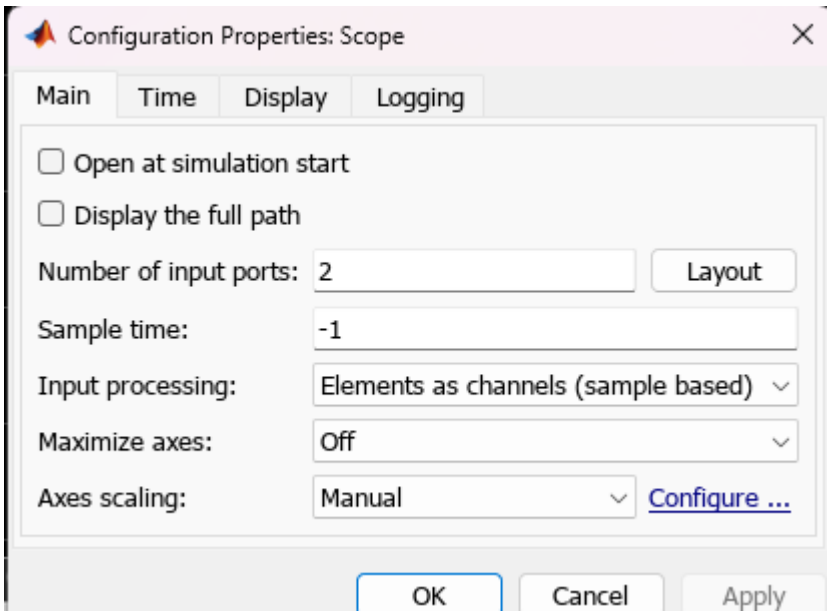

Block Parameters: Integrator_y1
Integrator
Continuous-time integration of the input signal.
Parameters
External reset: none
Initial condition source: internal
Initial condition:
1
Limit output
Wrap state
Show saturation port
Show state port
Absolute tolerance:
auto
Ignore limit and reset when linearizing
Enable zero-crossing detection
State Name: (e.g., 'position')
''
OK
Cancel
Help
Apply

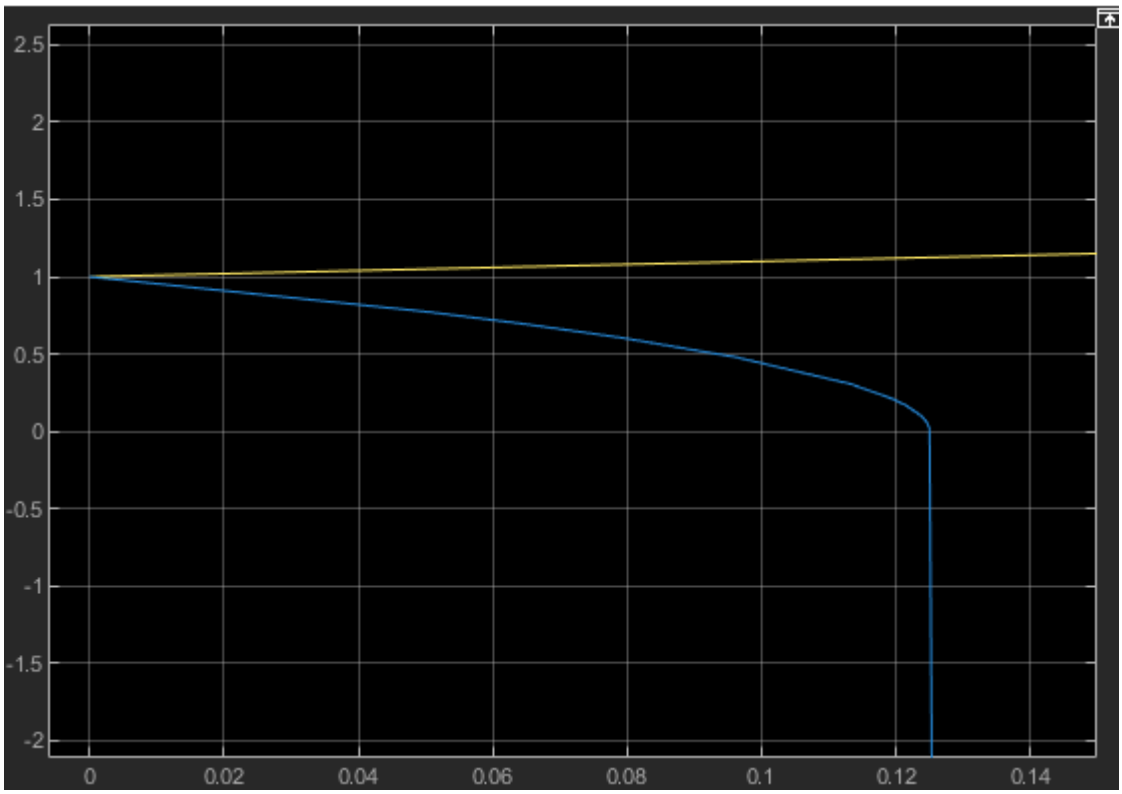

Block Parameters: Gain_x4
Gain
Element-wise gain (y = K.*u) or matrix gain (y = K*u or y = u*K).
Main
Signal Attributes
Parameter Attributes
Gain:
4
Multiplication: Element-wise(K.*u)
OK
Cancel
Help
Apply


Block Parameters: Product_y1
Product
Multiply or divide inputs. Choose element-wise or matrix product and specify one of the following:
a) * or / for each input port. For example, **/* performs the operation 'u1*u2/u3*u4'.
b) A scalar value specifies the number of input ports to be multiplied. For example, 2 performs the operation 'u1*u2'.
If there is only one input port and the Multiplication parameter is set to Element-wise(.*), a single * or / collapses the input signal using the specified operation. However, if the Multiplication parameter is set to Matrix(*), a single * causes the block to output the matrix unchanged, and a single / causes the block to output the matrix inverse.
Main
Signal Attributes
Number of inputs:
2
Multiplication:
Element-wise(.*)
OK
Cancel
Help
Apply


Block Parameters: Divided
Product
Multiply or divide inputs. Choose element-wise or matrix product and specify one of the following:
a) * or / for each input port. For example, **/* performs the operation 'u1*u2/u3*u4'.
b) A scalar value specifies the number of input ports to be multiplied. For example, 2 performs the operation 'u1*u2'.
If there is only one input port and the Multiplication parameter is set to Element-wise(.*), a single * or / collapses the input signal using the specified operation. However, if the Multiplication parameter is set to Matrix(*), a single * causes the block to output the matrix unchanged, and a single / causes the block to output the matrix inverse.
Main
Signal Attributes
Number of inputs:
*/
Multiplication:
Element-wise(.*)
OK
Cancel
Help
Apply


Block Parameters: Clock
Clock
Output the current simulation time.
Parameters
Display time
Decimation:
10
OK
Cancel
Help
Apply





Построение графиков непрерывной линейной модели решения системы ДУ:

Код:

```
% Пункт 4. Решение системы линейных ДУ
% dx/dt = 2x + 3y
% dy/dt = x + 4y
% с начальными условиями x(0)=1, y(0)=0

% 1. Задаем матрицу коэффициентов A
A = [2 3;
```

```
     1 4];

% 2. Задаем временной интервал и начальные условия
tspan = [0 5]; % Время от 0 до 5 секунд
init_cond = [1; 0]; % Вектор [x(0); y(0)]

% 3. Решаем систему ОДУ с помощью решателя ode45
[t, XY] = ode45(@(t, xy) A * xy, tspan, init_cond);

% 4. Извлекаем решения x(t) и y(t)
x = XY(:, 1);
y = XY(:, 2);

% 5. Строим графики
figure;
plot(t, x, 'b-', 'LineWidth', 2);
hold on;
plot(t, y, 'r--', 'LineWidth', 2);
grid on;
title('Решение системы линейных ДУ');
xlabel('Время t');
ylabel('x(t), y(t)');
legend('x(t)', 'y(t)');
hold off;

% 6. Дополнительный график для крупного плана в начале
figure;
plot(t, x, 'b-', t, y, 'r--', 'LineWidth', 2);
xlim([0 0.5]); % Смотрим на начало процесса
grid on;
title('Начальный участок решения системы ДУ');
xlabel('Время t');
ylabel('x(t), y(t)');
legend('x(t)', 'y(t)');
```

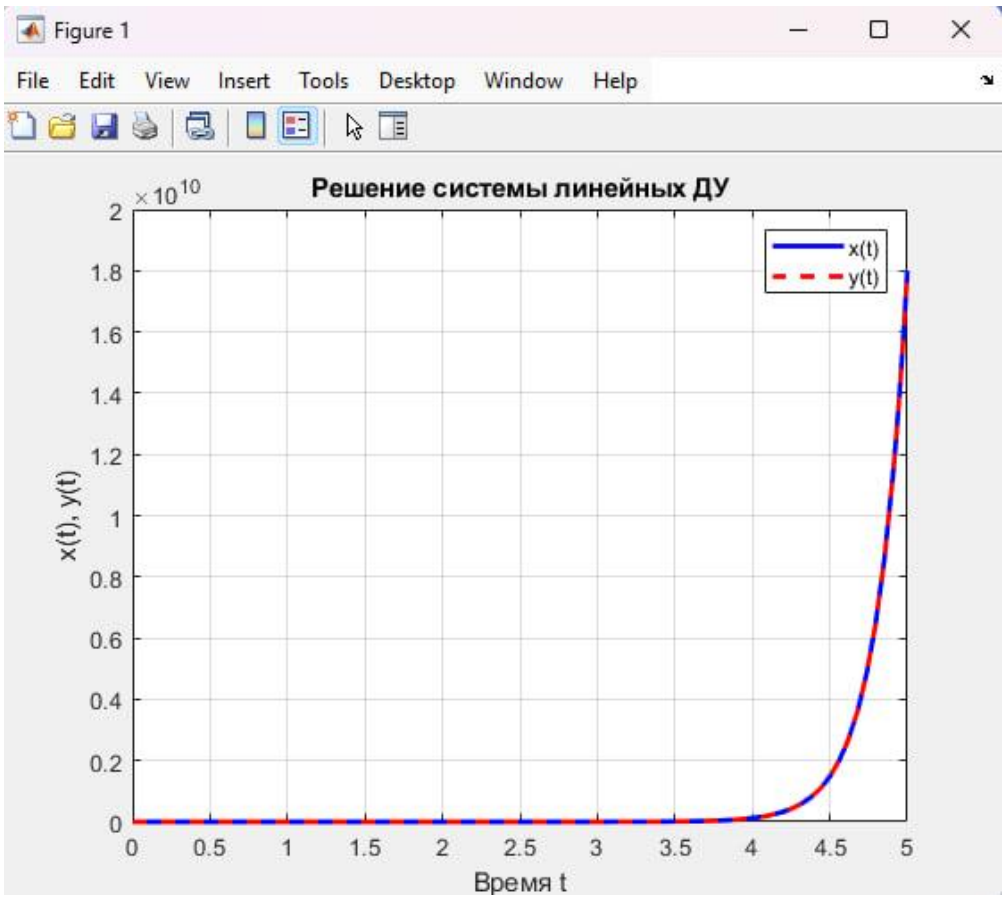


Figure 1
File Edit View Insert Tools Desktop Window Help
Решение системы линейных ДУ
×10^10
2
1.8
1.6
1.4
1.2
1
0.8
0.6
0.4
0.2
0
x(t), y(t)
0 0.5 1 1.5 2 2.5 3 3.5 4 4.5 5
Время t
x(t)
y(t)

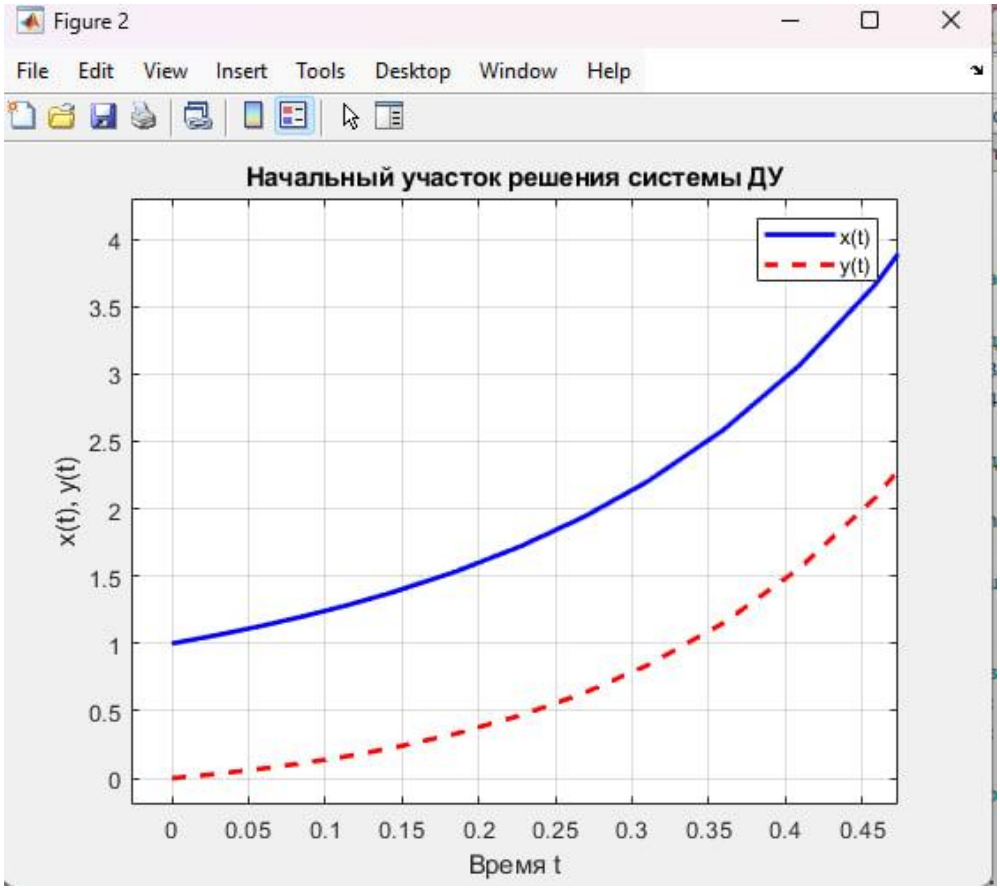
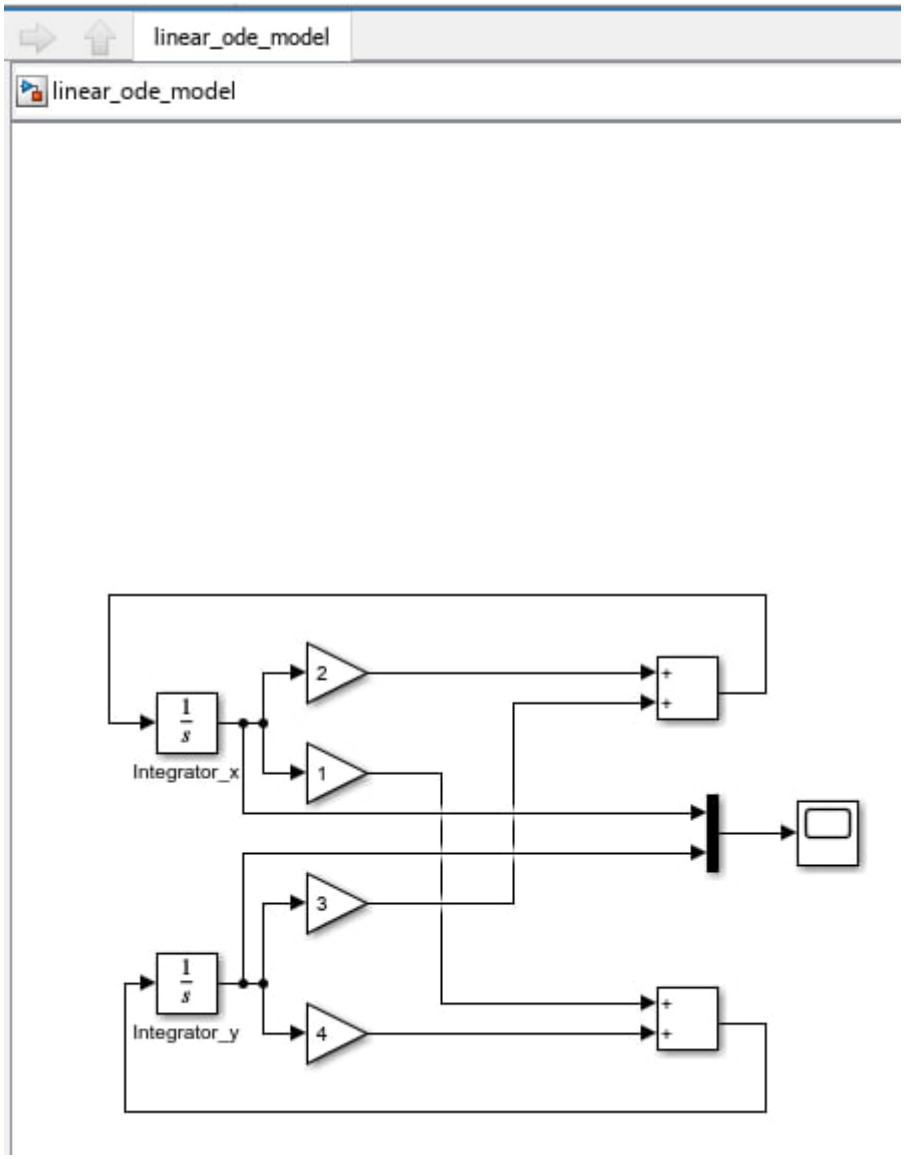
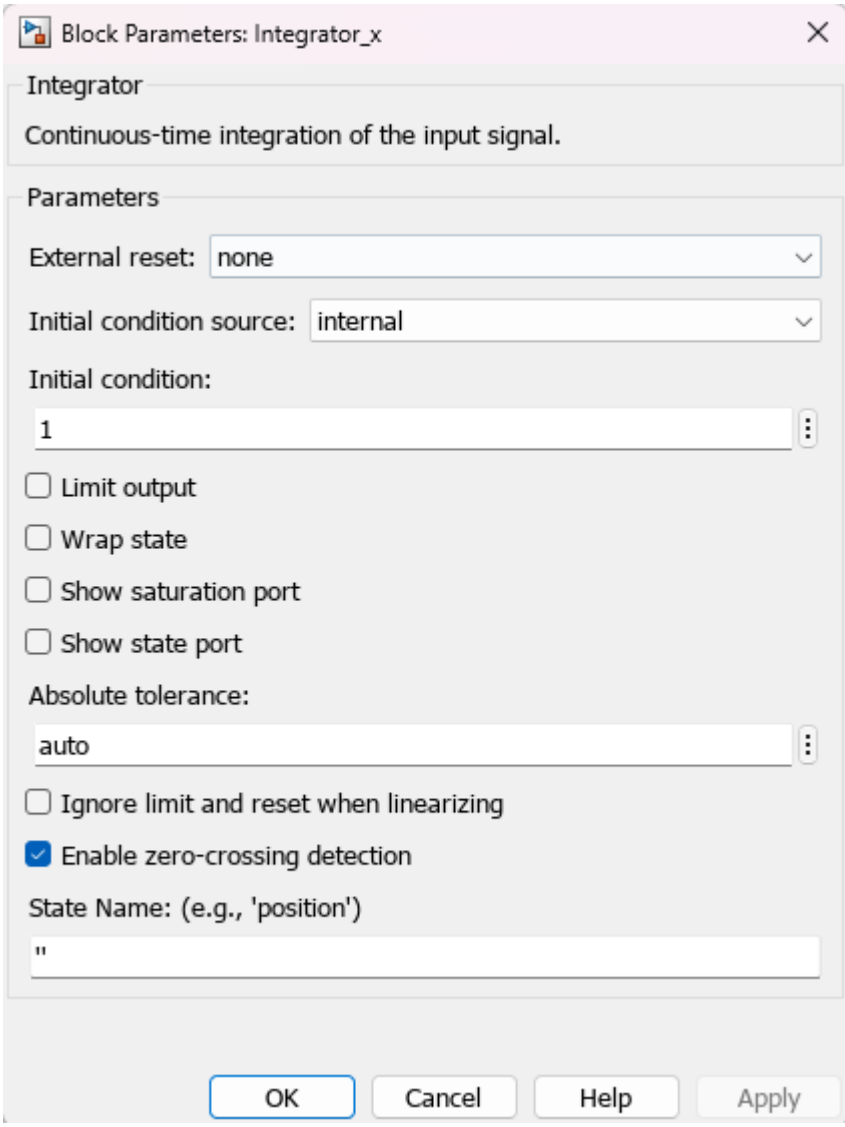
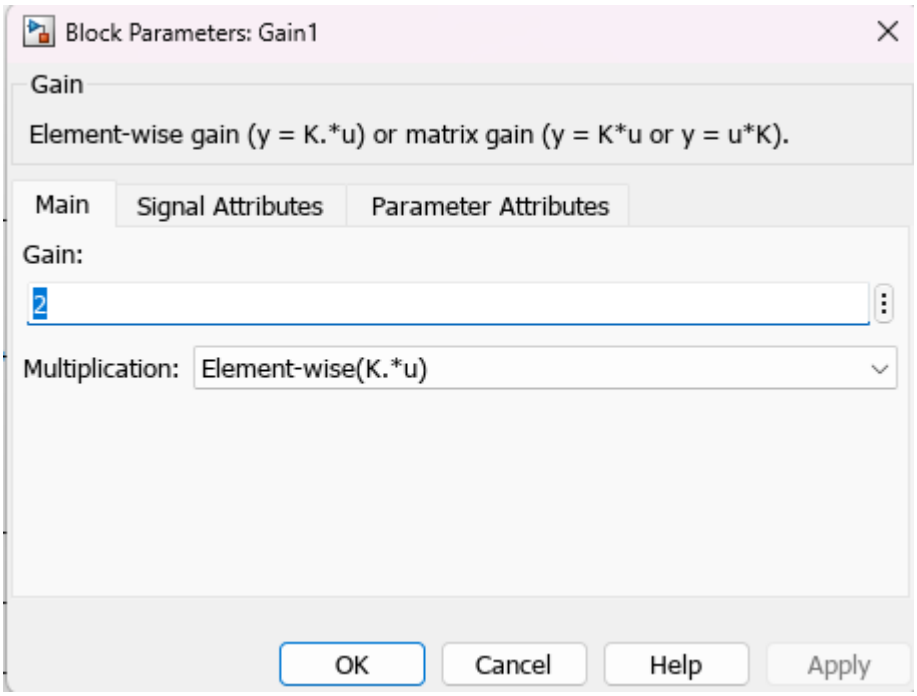


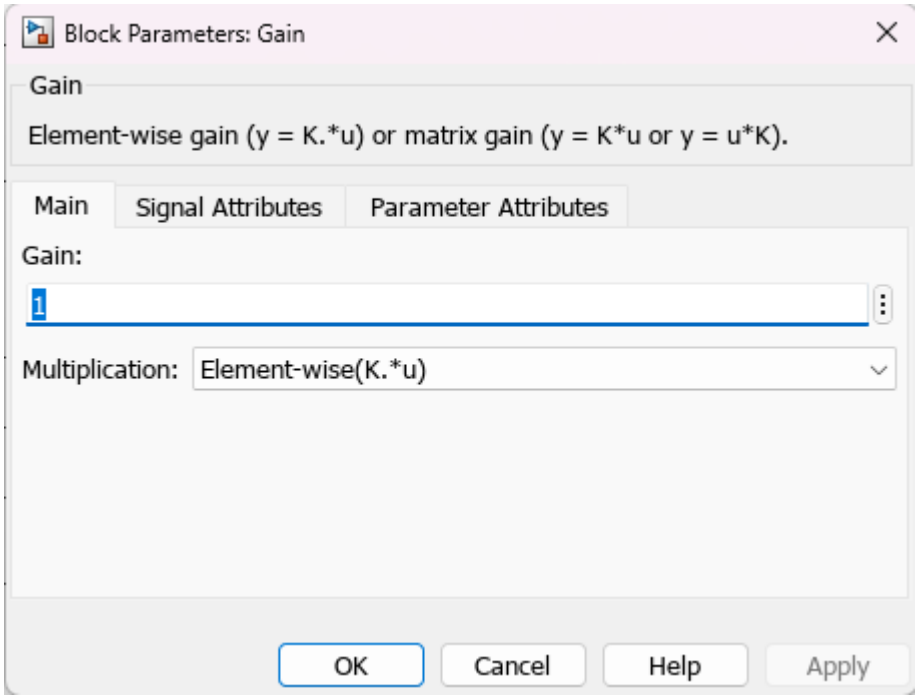
Figure 2
File Edit View Insert Tools Desktop Window Help
Начальный участок решения системы ДУ
4
3.5
3
2.5
2
1.5
1
0.5
0
x(t), y(t)
0 0.05 0.1 0.15 0.2 0.25 0.3 0.35 0.4 0.45
Время t
x(t)
y(t)

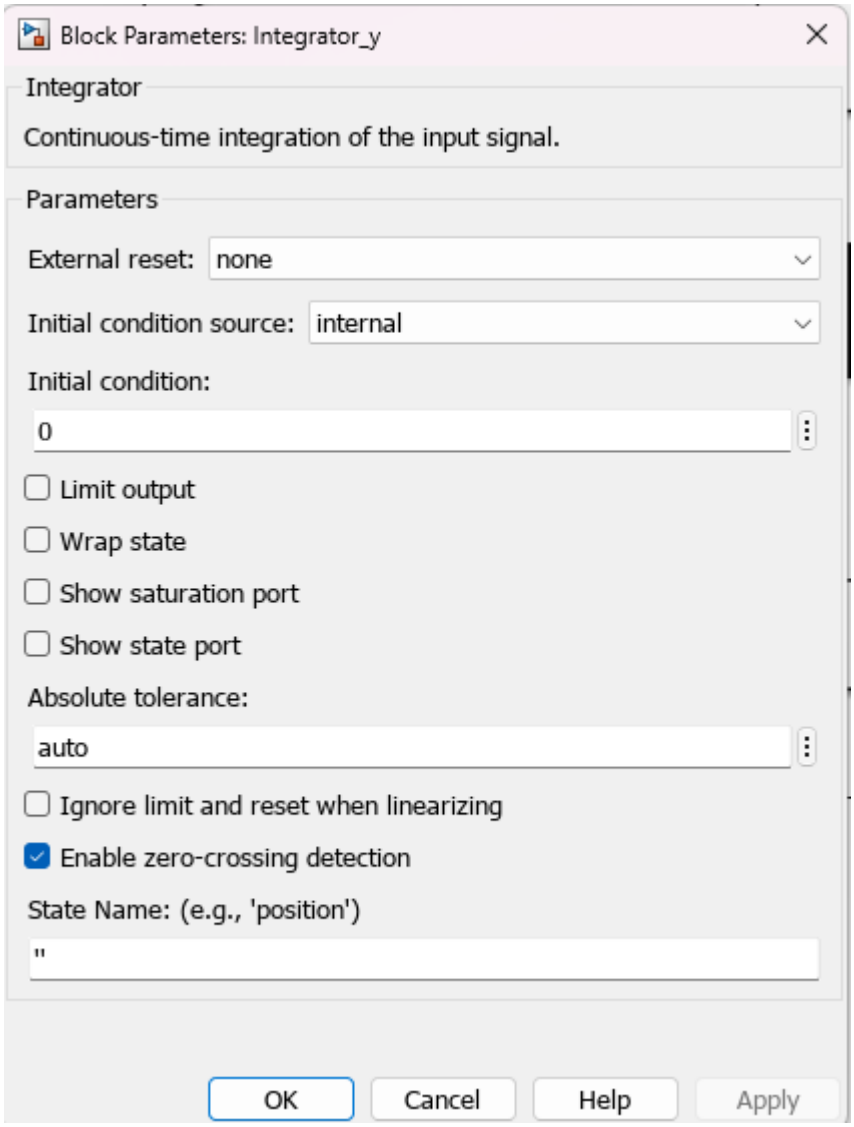
Разработка моделей Simulink для решения системы ДУ:

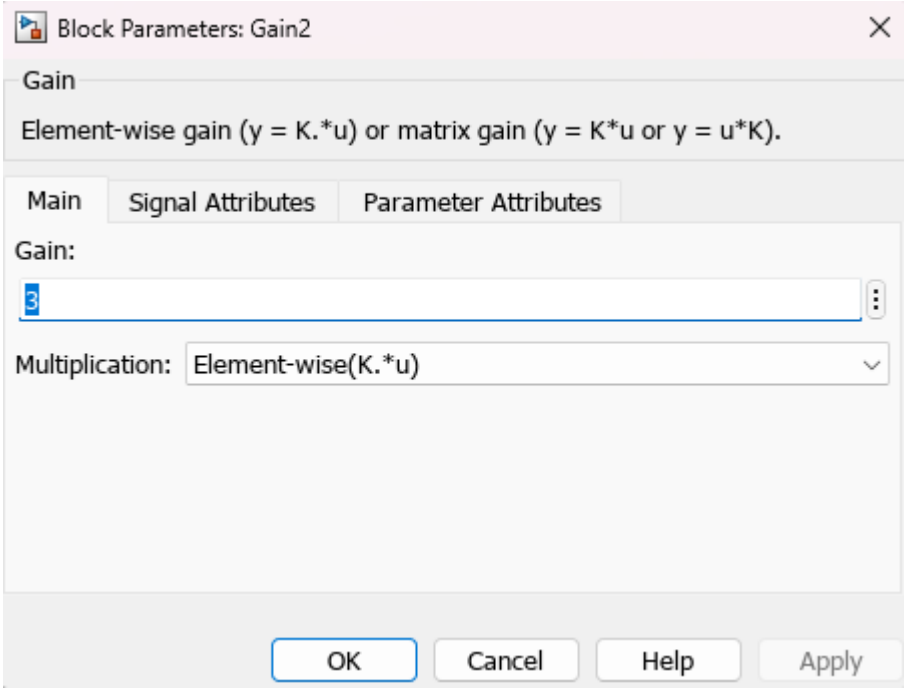


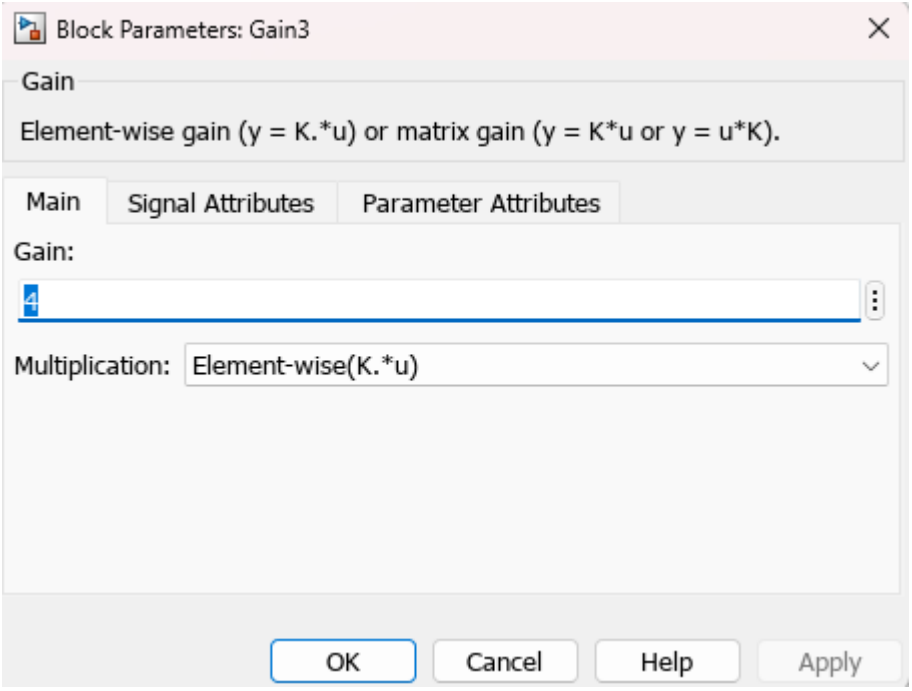

Block Parameters: Integrator_x
Integrator
Continuous-time integration of the input signal.
Parameters
External reset:
none
Initial condition source:
internal
Initial condition:
1
Limit output
Wrap state
Show saturation port
Show state port
Absolute tolerance:
auto
Ignore limit and reset when linearizing
Enable zero-crossing detection
State Name: (e.g., 'position')
''
OK
Cancel
Help
Apply

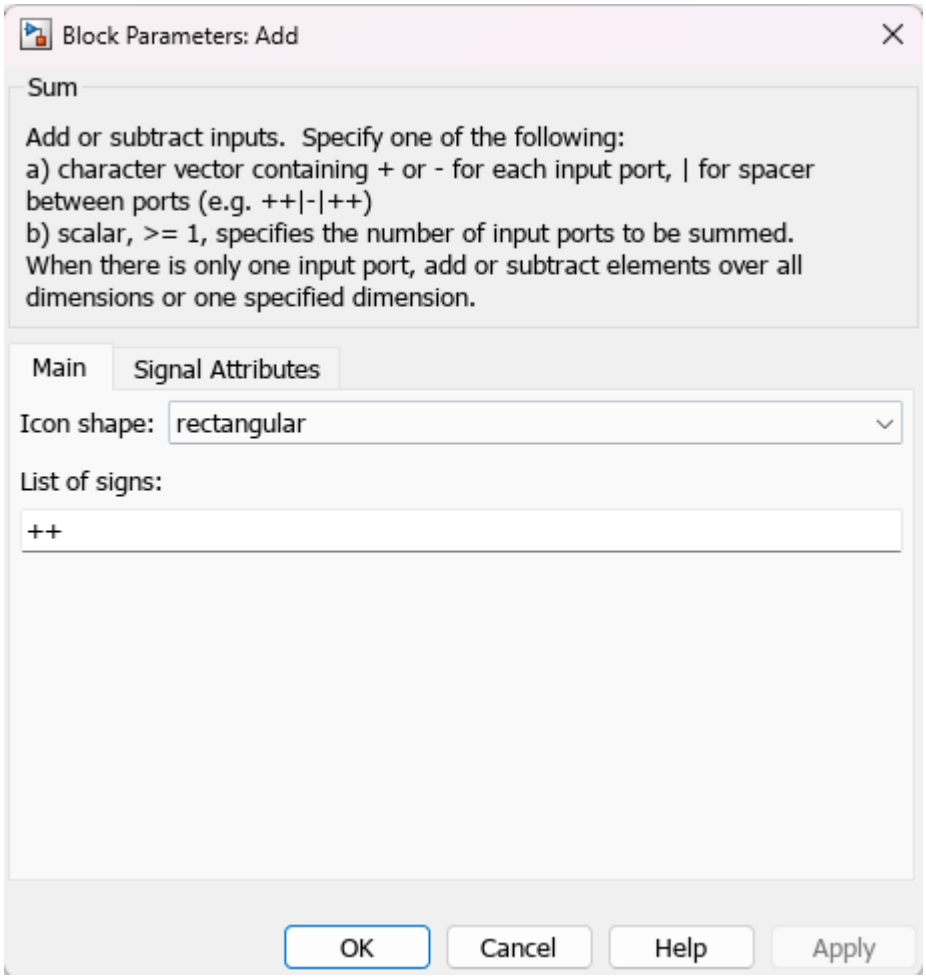

Block Parameters: Gain1
Gain
Element-wise gain (y = K.*u) or matrix gain (y = K*u or y = u*K).
Main
Signal Attributes
Parameter Attributes
Gain:
2
Multiplication:
Element-wise(K.*u)
OK
Cancel
Help
Apply

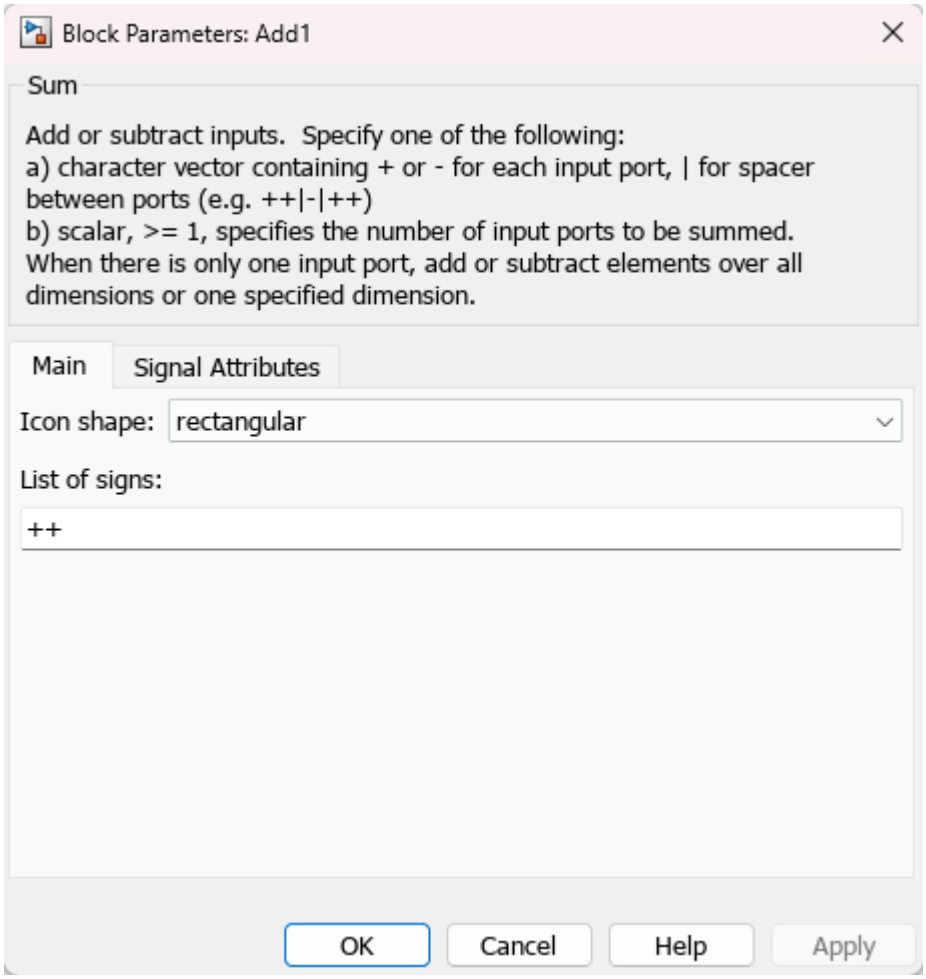

Block Parameters: Gain
Gain
Element-wise gain (y = K.*u) or matrix gain (y = K*u or y = u*K).
Main
Signal Attributes
Parameter Attributes
Gain:
1
Multiplication:
Element-wise(K.*u)
OK
Cancel
Help
Apply

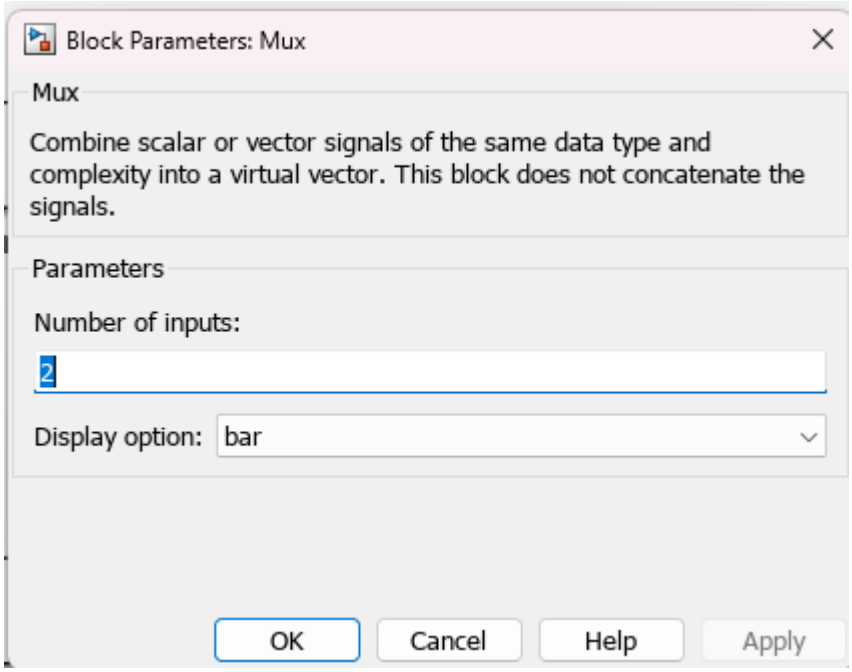

Block Parameters: Integrator_y
Integrator
Continuous-time integration of the input signal.
Parameters
External reset:
none
Initial condition source:
internal
Initial condition:
0
Limit output
Wrap state
Show saturation port
Show state port
Absolute tolerance:
auto
Ignore limit and reset when linearizing
Enable zero-crossing detection
State Name: (e.g., 'position')
''
OK
Cancel
Help
Apply

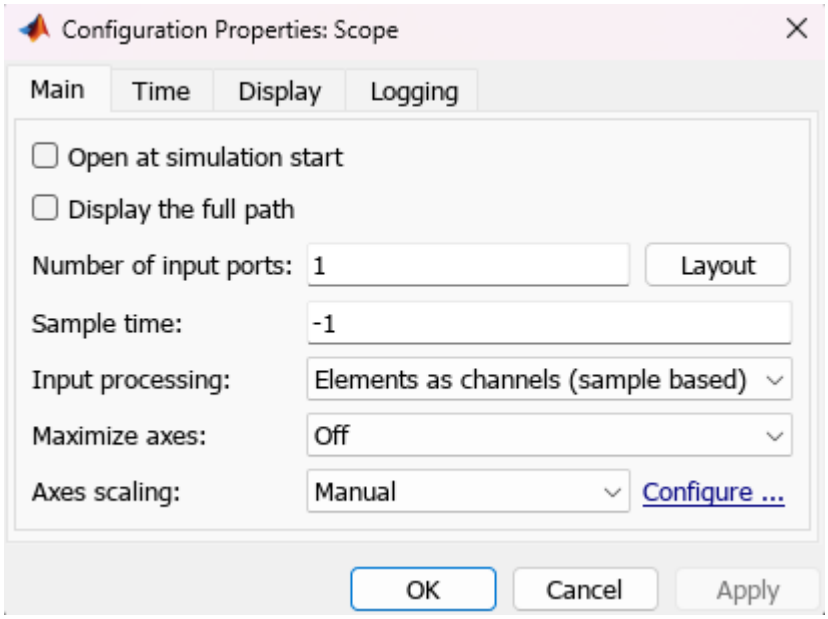

Block Parameters: Gain2
Gain
Element-wise gain (y = K.*u) or matrix gain (y = K*u or y = u*K).
Main
Signal Attributes
Parameter Attributes
Gain:
3
Multiplication:
Element-wise(K.*u)
OK
Cancel
Help
Apply

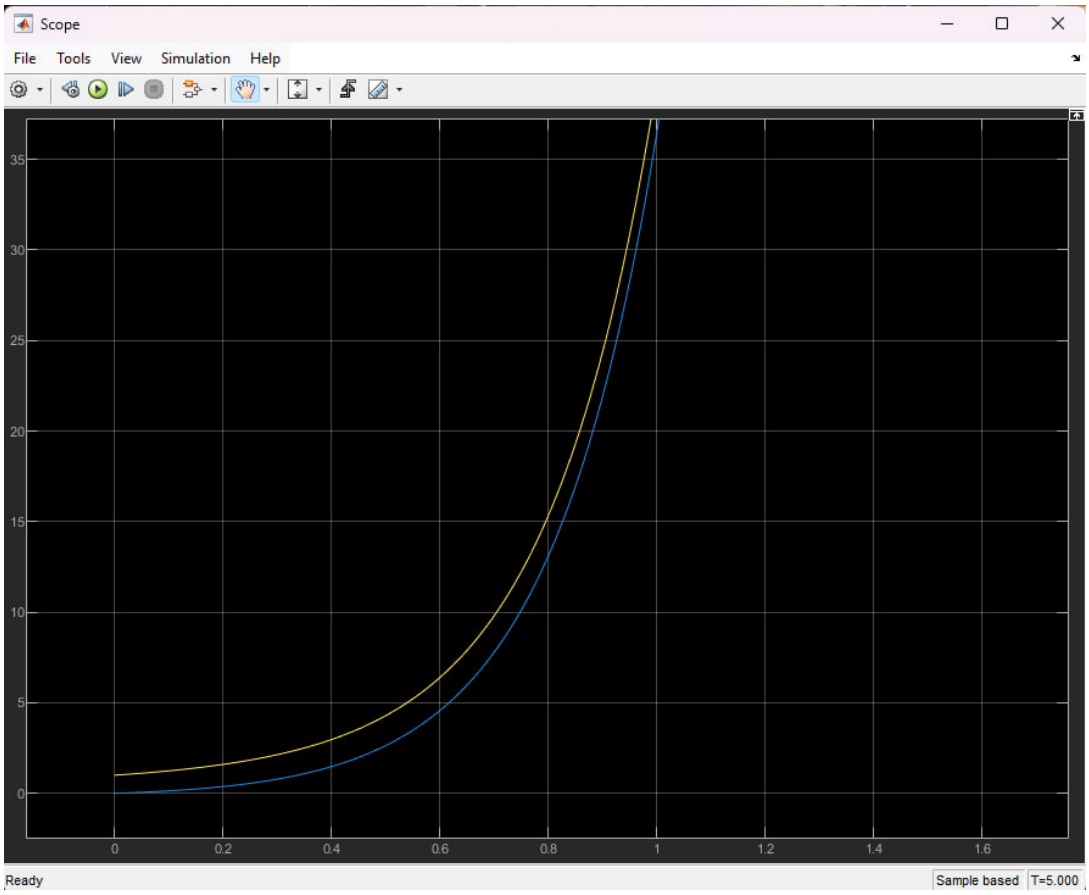

Block Parameters: Gain3
Gain
Element-wise gain (y = K.*u) or matrix gain (y = K*u or y = u*K).
Main
Signal Attributes
Parameter Attributes
Gain:
4
Multiplication:
Element-wise(K.*u)
OK
Cancel
Help
Apply

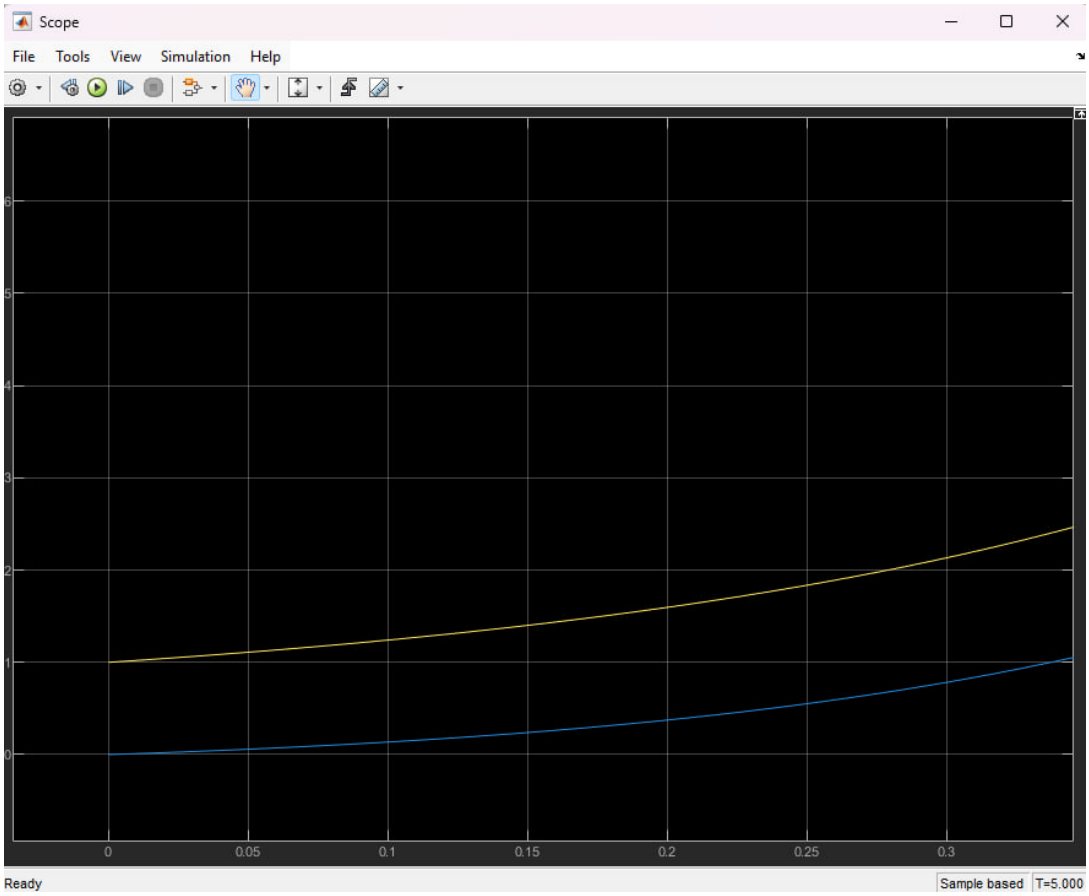

Block Parameters: Add
Sum
Add or subtract inputs. Specify one of the following:
a) character vector containing + or - for each input port, | for spacer between ports (e.g. ++|-|++)
b) scalar, >= 1, specifies the number of input ports to be summed.
When there is only one input port, add or subtract elements over all dimensions or one specified dimension.
Main
Signal Attributes
Icon shape:
rectangular
List of signs:
++
OK
Cancel
Help
Apply


Block Parameters: Add1
Sum
Add or subtract inputs. Specify one of the following:
a) character vector containing + or - for each input port, | for spacer between ports (e.g. ++|-|++)
b) scalar, >= 1, specifies the number of input ports to be summed.
When there is only one input port, add or subtract elements over all dimensions or one specified dimension.
Main
Signal Attributes
Icon shape:
rectangular
List of signs:
++
OK
Cancel
Help
Apply


Block Parameters: Mux
Mux
Combine scalar or vector signals of the same data type and complexity into a virtual vector. This block does not concatenate the signals.
Parameters
Number of inputs:
2
Display option:
bar
OK
Cancel
Help
Apply


Configuration Properties: Scope
Main
Time
Display
Logging
Open at simulation start
Display the full path
Number of input ports:
1
Layout
Sample time:
-1
Input processing:
Elements as channels (sample based)
Maximize axes:
Off
Axes scaling:
Manual
Configure ...
OK
Cancel
Apply


Scope
File
Tools
View
Simulation
Help
35
30
25
20
15
10
5
0
0
0.2
0.4
0.6
0.8
1
1.2
1.4
1.6
Ready
Sample based
T=5.000

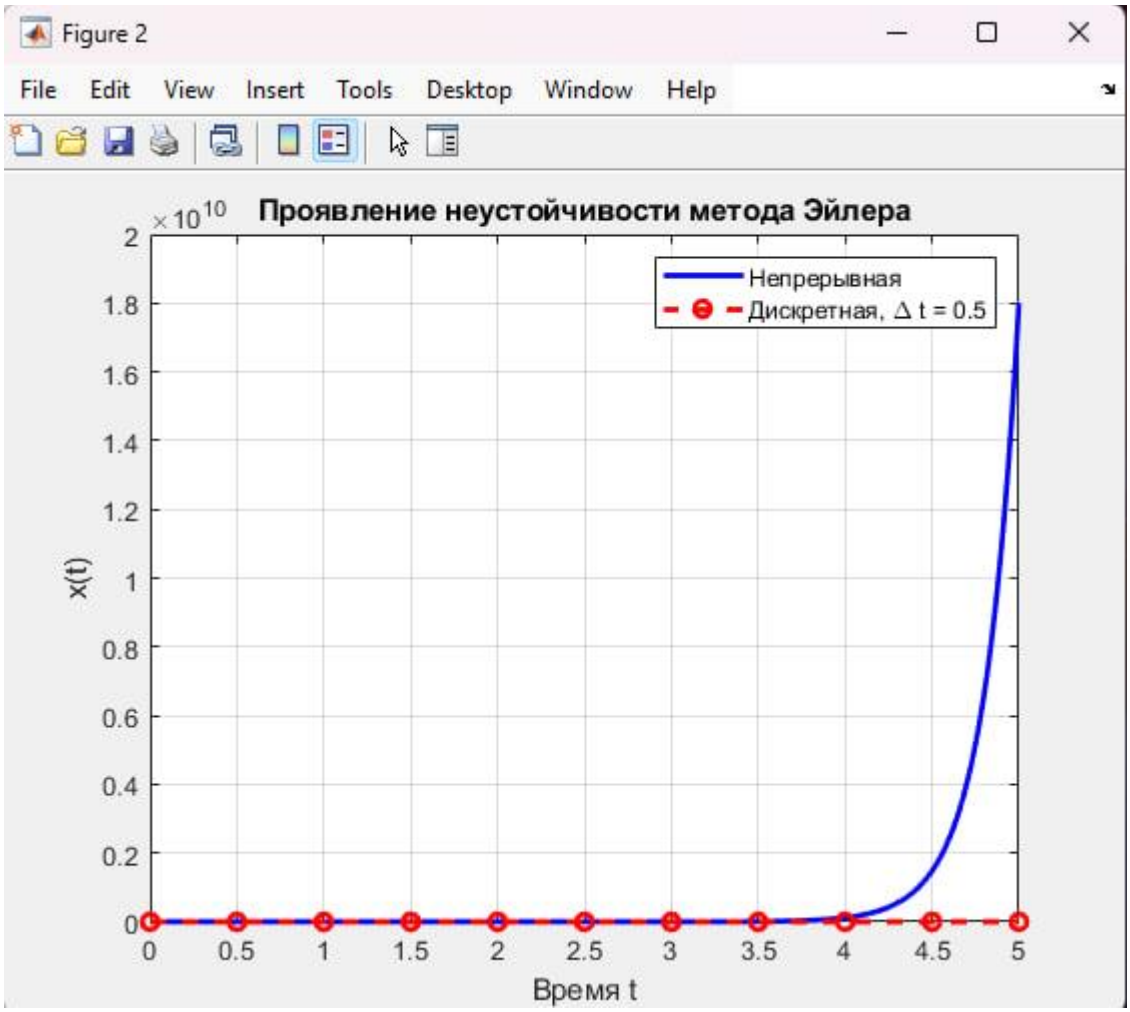


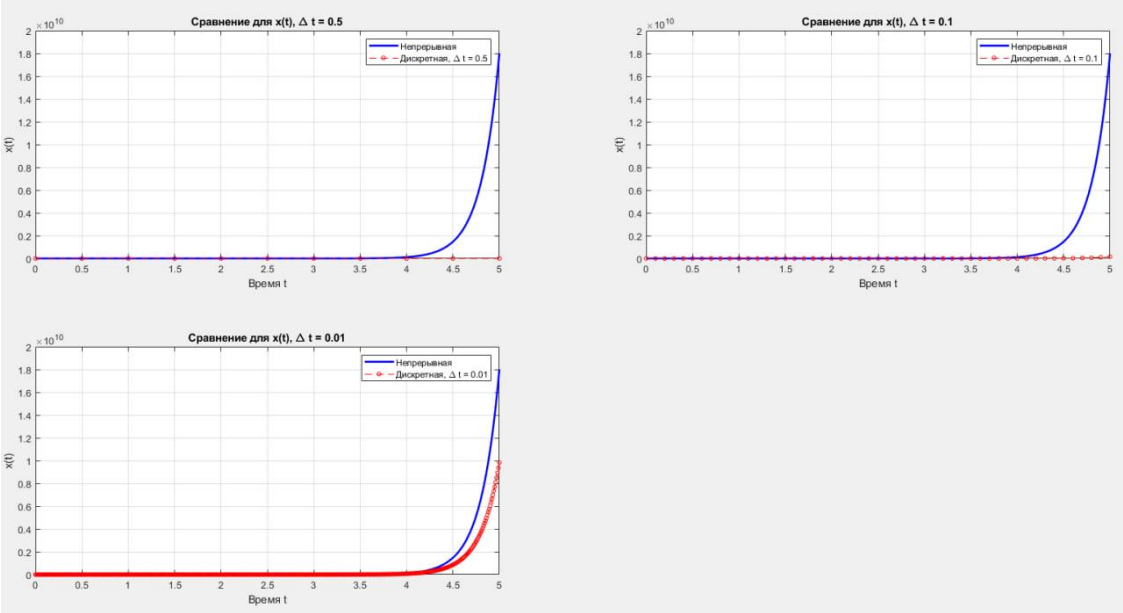
Дискретизация системы и сравнительный анализ:

```
% Пункт 6. Сравнение непрерывной и дискретной моделей

% Параметры системы
A = [2 3; 1 4];
init_cond = [1; 0];
tspan = [0 5];
t_continuous = linspace(tspan(1), tspan(2), 1000); % Вектор времени для точного
решения

% 1. Точное решение (численное, через ode45)
[~, XY_cont] = ode45(@(t, xy) A * xy, t_continuous, init_cond);
x_cont = XY_cont(:, 1);
y_cont = XY_cont(:, 2);

% 2. Дискретное решение методом Эйлера с разными шагами
dt_values = [0.5, 0.1, 0.01]; % Шаги дискретизации

figure;

for i = 1:length(dt_values)
    dt = dt_values(i);
    t_discrete = tspan(1):dt:tspan(2); % Вектор времени для дискретного решения

    % Инициализация
    x_disc = zeros(size(t_discrete));
    y_disc = zeros(size(t_discrete));
```

```
    x_disc(1) = init_cond(1);
    y_disc(1) = init_cond(2);

    % Метод Эйлера
    for k = 1:length(t_discrete)-1
        dx_dt = 2*x_disc(k) + 3*y_disc(k);
        dy_dt = 1*x_disc(k) + 4*y_disc(k);

        x_disc(k+1) = x_disc(k) + dt * dx_dt;
        y_disc(k+1) = y_disc(k) + dt * dy_dt;
    end

    % Строим графики для x(t)
    subplot(2, 2, i);
    plot(t_continuous, x_cont, 'b-', 'LineWidth', 2, 'DisplayName',
'Непрерывная');
    hold on;
    plot(t_discrete, x_disc, 'ro--', 'MarkerSize', 4, 'DisplayName',
['Дискретная, \Delta t = ', num2str(dt)]);
    hold off;
    grid on;
    title(['Сравнение для x(t), \Delta t = ', num2str(dt)]);
    xlabel('Время t');
    ylabel('x(t)');
    legend;

    % Для y(t) на отдельном рисунке
%     subplot(2, 2, i+3);
%     plot(t_continuous, y_cont, 'r-', 'LineWidth', 2, 'DisplayName',
'Непрерывная');
%     hold on;
%     plot(t_discrete, y_disc, 'go--', 'MarkerSize', 4, 'DisplayName',
['Дискретная, \Delta t = ', num2str(dt)]);
%     hold off;
%     grid on;
%     title(['Сравнение для y(t), \Delta t = ', num2str(dt)]);
%     xlabel('Время t');
%     ylabel('y(t)');
%     legend;
end

% 3. Отдельный график для демонстрации неустойчивости при большом dt
dt_bad = 0.5;
t_bad = tspan(1):dt_bad:tspan(2);
x_bad = zeros(size(t_bad));
y_bad = zeros(size(t_bad));
x_bad(1) = init_cond(1);
y_bad(1) = init_cond(2);

for k = 1:length(t_bad)-1
    dx_dt = 2*x_bad(k) + 3*y_bad(k);
    dy_dt = 1*x_bad(k) + 4*y_bad(k);

    x_bad(k+1) = x_bad(k) + dt_bad * dx_dt;
    y_bad(k+1) = y_bad(k) + dt_bad * dy_dt;
end

figure;
plot(t_continuous, x_cont, 'b-', 'LineWidth', 2, 'DisplayName', 'Непрерывная');
hold on;
plot(t_bad, x_bad, 'ro--', 'LineWidth', 2, 'MarkerSize', 6, 'DisplayName',
['Дискретная, \Delta t = ', num2str(dt_bad)]);
```

```
hold off;
grid on;
title('Проявление неустойчивости метода Эйлера');
xlabel('Время t');
ylabel('x(t)');
legend;
```





Вывод:

В ходе выполнения лабораторной работы я освоил основные приёмы моделирования непрерывных процессов в среде MatLab Simulink. Я научился преобразовывать дифференциальные уравнения высших порядков в системы уравнений первого порядка, строить для них структурные схемы из базовых блоков (интеграторов, усилителей, сумматоров и др.), корректно задавать начальные условия и параметры решателя. На практике я реализовал модели как для нелинейного уравнения второго порядка, так и для системы линейных дифференциальных уравнений, проанализировал их поведение и визуализировал результаты. Кроме того, я изучил метод дискретизации Эйлера, создал модель для разностной системы и провёл сравнительный анализ точности непрерывной и дискретной моделей при различных шагах дискретизации, что позволило мне понять ограничения и особенности численных методов решения дифференциальных уравнений.